Rodrigo Fernandez

Senior .NET Backend Engineer
Katy, Texas | +1 (407) 801-1287 | https://www.linkedin.com/in/rodrigo-bermejo-fern%C3%A1ndez-9a5a0664/| rodrigobfdev@gmail.com

Summary

Senior **.NET Backend Engineer** with deep experience across cloud services, SaaS platforms, fintech-style billing workflows, healthcare data integrations, and enterprise modernization, building resilient APIs with **C# 8–12**, **ASP.NET Core 3.1–8**, **.NET Framework 4.6.1–4.8**, **EF Core 3.1–8**, SQL Server, PostgreSQL, Azure, AWS, Docker, and message-driven architecture; strong background in migrating legacy services to modern LTS runtimes, fixing production latency, memory, and transaction consistency issues, improving reliability through observability, automated testing, and CI/CD, and delivering secure backend systems for high-volume business operations.

Skills

- **Backend: C# 8–12**, **ASP.NET Core 3.1–8**, **.NET Core 3.1–8**, **.NET Framework 4.6.1–4.8**, Web API, REST, gRPC, Minimal APIs, background workers, Windows Services
- **Database: SQL Server 2016–2022**, PostgreSQL 11–16, Entity Framework Core 3.1–8, Dapper, T-SQL, stored procedures, indexing, query tuning, migrations
- **Cloud & DevOps:** Azure App Service, Azure Functions, Azure Service Bus, AWS EC2, S3, Lambda, CloudWatch, Docker, Kubernetes basics, GitHub Actions, Azure DevOps, CI/CD
- **Architecture:** Clean Architecture, Domain-Driven Design, CQRS, repository pattern, unit of work, microservices, message queues, event-driven systems, API versioning
- **Security & Quality:** OAuth2, OIDC, JWT, RBAC, Serilog, Application Insights, OpenTelemetry basics, xUnit, NUnit, Moq, integration testing, SonarQube

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Architected **ASP.NET Core 6–8** backend services for cloud hosting and customer operations, separating billing, provisioning, and identity domains so release scope stayed controlled across high-volume SaaS workflows.
- Migrated legacy **.NET Core 3.1** APIs to **.NET 6–8 LTS**, resolving package incompatibilities, nullable reference warnings, middleware ordering errors, and authentication regression issues before production rollout.
- Reduced API response time by **38%** by replacing over-fetching Entity Framework queries with compiled queries, pagination contracts, filtered includes, and tuned SQL Server indexes.
- Improved background job reliability by **31%** using **Azure Service Bus**, Hangfire, retry policies, idempotency keys, and poison-message handling for provisioning and invoice workflows.
- Solved intermittent **DbContext concurrency** errors by moving scoped services out of singleton workers, adding transaction boundaries, and improving async disposal behavior across hosted services.
- Implemented secure REST APIs with **JWT**, OAuth2/OIDC validation, role-based policies, request signing, and centralized authorization handlers for multi-tenant customer administration.
- Built observability around **Application Insights**, CloudWatch, structured Serilog logs, metrics, and correlation IDs, making production incident tracing faster across API, worker, and database layers.

- Refactored monolithic backend modules into clean service boundaries with repository, unit-of-work, mediator, and domain-service patterns while keeping the migration compatible with active releases.
- Stabilized CI/CD by adding **GitHub Actions**, Docker build caching, unit tests, integration tests, migration checks, and rollback scripts, cutting failed deployments by **27%**.
- Resolved memory pressure in high-throughput file-processing APIs by streaming payloads, avoiding large byte-array buffering, tuning GC-sensitive allocations, and adding upload-size guards.
- Delivered new subscription, audit-log, and customer-notification features by coordinating product requirements with backend contracts, database migrations, and API versioning strategy.
- Mentored engineers on **C# 10–12**, async patterns, dependency injection, secure configuration, and production troubleshooting so backend delivery stayed consistent across distributed teams.

NIX United —Software Engineer *(Oct 2016 – Mar 2020)*

- Developed enterprise backend APIs with **ASP.NET Core 2.1–3.1**, **.NET Framework 4.6.1–4.8**, SQL Server, RabbitMQ, and Azure DevOps for finance, logistics, and internal workflow platforms.
- Migrated selected Web API and WCF modules toward **.NET Core 3.1** by introducing **.NET Standard 2.0** shared libraries, dependency injection, configuration providers, and compatibility wrappers.
- Reduced nightly batch failure rate by **29%** after redesigning retry logic, SQL timeout handling, exception classification, and alerting around payment reconciliation and document synchronization jobs.
- Fixed recurring SQL deadlocks by analyzing execution plans, narrowing transaction scope, adding covering indexes, and changing repository methods that held locks longer than required.
- Implemented message-driven integrations with RabbitMQ exchanges, durable queues, dead-letter routing, and idempotent consumers so external partner failures no longer blocked core user workflows.
- Built EF Core and SQL migration procedures with rollback scripts, seed-data validation, and release checklists that helped teams deploy schema changes more safely across environments.
- Strengthened API security with **JWT**, refresh-token rotation, role-based access rules, password hashing improvements, and audit trails for sensitive customer and administrator operations.
- Increased automated backend coverage by **34%** using xUnit, Moq, integration tests, and test database containers around controllers, services, repositories, and background workers.
- Resolved serialization and timezone defects in distributed APIs by standardizing UTC storage, explicit JSON contracts, contract tests, and backward-compatible DTO versioning.
- Supported Azure DevOps pipelines with build validation, NuGet package publishing, environment variables, secure secrets, and approval gates for staging and production releases.
- Worked flexibly across client domains by adapting **C#**, SQL, REST, messaging, and cloud deployment patterns to each system's reliability, compliance, and performance requirements.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built backend modules for customer portals, CMS platforms, and e-commerce systems using **ASP.NET MVC 5**, **Web API 2**, **.NET Framework 4.5–4.6**, SQL Server, and IIS.
- Upgraded older **.NET Framework 4.0–4.5** applications to **.NET Framework 4.5.2–4.6**, fixing NuGet conflicts, binding redirects, web.config issues, and deprecated library usage.
- Improved checkout and order-processing performance by **22%** through SQL index tuning, caching frequently used catalog data, and reducing unnecessary database round trips.
- Fixed session expiration and cookie-authentication bugs by reviewing IIS configuration, machine keys, timeout settings, browser cookie behavior, and inconsistent login-state handling.
- Implemented payment callback, invoice, email-notification, and admin-reporting features with clear API contracts, validation layers, and defensive error handling for failed provider responses.
- Created stored procedures, views, and Dapper-based data access methods for reporting workflows where Entity Framework queries were too slow or difficult to optimize.

- Added server-side caching and cache invalidation rules for product, category, and configuration data, reducing repeated reads during peak traffic periods by **26%**.
- Introduced structured logging, custom exception filters, and user-friendly error pages so production issues could be diagnosed without exposing sensitive technical details.
- Coordinated with frontend developers by defining REST payloads, status codes, validation messages, and backward-compatible endpoint changes before each client release.
- Improved deployment stability by documenting IIS setup, app pool settings, connection strings, package publishing steps, and manual rollback procedures for smaller production environments.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Maintained internal admin systems with **C# 4–5**, **ASP.NET MVC 3–4**, Web Forms, ADO.NET, SQL Server, and IIS while learning production support discipline from senior engineers.
- Implemented CRUD screens, reporting filters, export functions, and permission checks for business users who needed faster access to customer, inventory, and operations data.
- Resolved frequent null-reference and database-connection errors by adding validation, safer mapping logic, using-block cleanup, and clearer exception handling around legacy data access code.
- Improved report execution time by **18%** after simplifying stored procedures, adding missing indexes, and reducing repeated queries inside loop-based server-side processing.
- Assisted with gradual migration from Web Forms pages to **ASP.NET MVC 4** by separating presentation logic, shared validation rules, and reusable service classes.
- Wrote SQL scripts, stored procedures, and small maintenance jobs to clean duplicate records, normalize reference data, and support operational reporting requirements.
- Added form validation, authorization checks, and safer input handling to reduce user-entry errors and strengthen protection around internal administrative workflows.
- Supported IIS deployments by updating web.config values, verifying connection strings, checking Windows event logs, and confirming application pool behavior after releases.
- Built strong fundamentals in **C#**, object-oriented design, debugging, source control, and code review while contributing carefully to live business applications.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Contributed to internal engineering utilities using **C#**, **ASP.NET MVC 4**, JavaScript, and SQL Server patterns while learning enterprise development standards from senior engineers.
- Assisted with bug reproduction, log review, and small feature fixes for dashboard pages, improving reliability of internal tools used by technical support teams.
- Implemented minor UI updates and backend validation improvements under code review, gaining practical experience with accessibility, maintainability, and secure coding expectations.
- Prepared documentation for setup steps, known issues, and troubleshooting paths so future interns and team members could onboard more smoothly to internal applications.

Education

Texas State University
Bachelor's Degree in Computer Science (*Sep 2008 – May 2012)*